

CHECKPOINT 02 - PROMPTTOOLKIT

Comparação de Técnicas de Prompt Engineering com LLMs

INTEGRANTE

Lucas Mota Ferreira - RM566670

DISCIPLINA

Prompt Engineering & Artificial Intelligence

TOOLKIT

SentinelPrompt

DOMÍNIO

Toolkit de Prompt Engineering para E-Commerce

MODELO UTILIZADO

gpt-oss:120b via Ollama

1. DOMÍNIO E PROBLEMA

O domínio escolhido para o desenvolvimento deste projeto foi o atendimento ao cliente em e-commerce. Empresas de comércio digital recebem diariamente milhares de mensagens relacionadas a reclamações, atrasos em entregas, devoluções, suporte técnico, dúvidas de clientes e problemas com produtos. A análise manual desse grande volume de informações torna-se um processo caro, demorado e pouco escalável. Diante desse cenário, o projeto foi desenvolvido com o objetivo de demonstrar como técnicas de *Prompt Engineering* podem ser utilizadas para automatizar tarefas reais de análise textual por meio de *Large Language Models* (LLMs).

O SentinelPrompt foi desenvolvido para comparar automaticamente diferentes estratégias de prompting, com o objetivo de identificar qual técnica apresenta maior acurácia, menor custo em tokens, maior consistência nas respostas e melhor desempenho para cada tipo de tarefa. A proposta do toolkit é avaliar como diferentes abordagens de interação com *Large Language Models* influenciam diretamente a qualidade e a eficiência dos resultados obtidos. Entre as técnicas analisadas no projeto estão o *Zero-Shot Prompting*, *Few-Shot Prompting*, *Chain-of-Thought* e *Role Prompting*.

Tarefas implementadas:

Tarefa	Tipo	Objetivo
classificacao_sentimento	Classificação	Identificar sentimento de mensagens
classificacao_urgencia	Classificação	Identificar nível de urgência
extracao_dados	Extração	Extrair informações estruturadas em JSON

2. ARQUITETURA DO SISTEMA

Fluxo de execução:

CP02_LucasRM566670/

```
├── data/
├── docs/
├── output/
│   └── graficos/
├── prompts/
├── src/
│   ├── evaluator.py
│   ├── llm_client.py
│   ├── prompt_builder.py
│   ├── report.py
│   ├── tasks.py
│   └── techniques.py
├── main.py
├── requirements.txt
├── README.md
└── .env.example
```

Estrutura dos módulos:

Módulo	Responsabilidade
main.py	Orquestra o fluxo completo
techniques.py	Implementa as técnicas de prompting
prompt_builder.py	Constrói prompts dinamicamente
llm_client.py	Comunicação com o Ollama
evaluator.py	Métricas e avaliação
report.py	Geração de CSV e gráficos
tasks.py	Definição das tarefas

3. STACK TÉCNICA

Componente	Tecnologia
Linguagem	Python 3.14.5
Modelo	gpt-oss:120b
Plataforma	Ollama
Tokens	tiktoken
Visualização	matplotlib + pandas
Requisições HTTP	requests
Configuração	python-dotenv

Execução:

Instalação: `pip install -r requirements.txt`

Execução: `python main.py`

O sistema gera automaticamente: relatório CSV, gráficos, recomendações e análises comparativas.

4. RESULTADOS

Acurácia média por técnica:

Técnica	Sentimento	Urgência	Extração	Média Geral
Zero-Shot	80%	100%	65%	81.7%
Few-Shot	80%	100%	85%	88.3%
Chain-of-Thought	80%	100%	75%	85.0%
Role Prompting	80%	100%	65%	81.7%

Os resultados indicaram que o Few-Shot apresentou o melhor desempenho geral, enquanto o Chain-of-Thought se destacou pela contextualização das respostas. O Zero-Shot Prompting obteve o menor custo computacional, e todas as técnicas alcançaram 100% de acerto na classificação de urgência das mensagens analisadas.

5. GRÁFICOS GERADOS

Acurácia média por técnica: O gráfico demonstra que Few-Shot obteve melhor desempenho em extração estruturada, enquanto todas as técnicas tiveram desempenho semelhante em classificação de urgência.

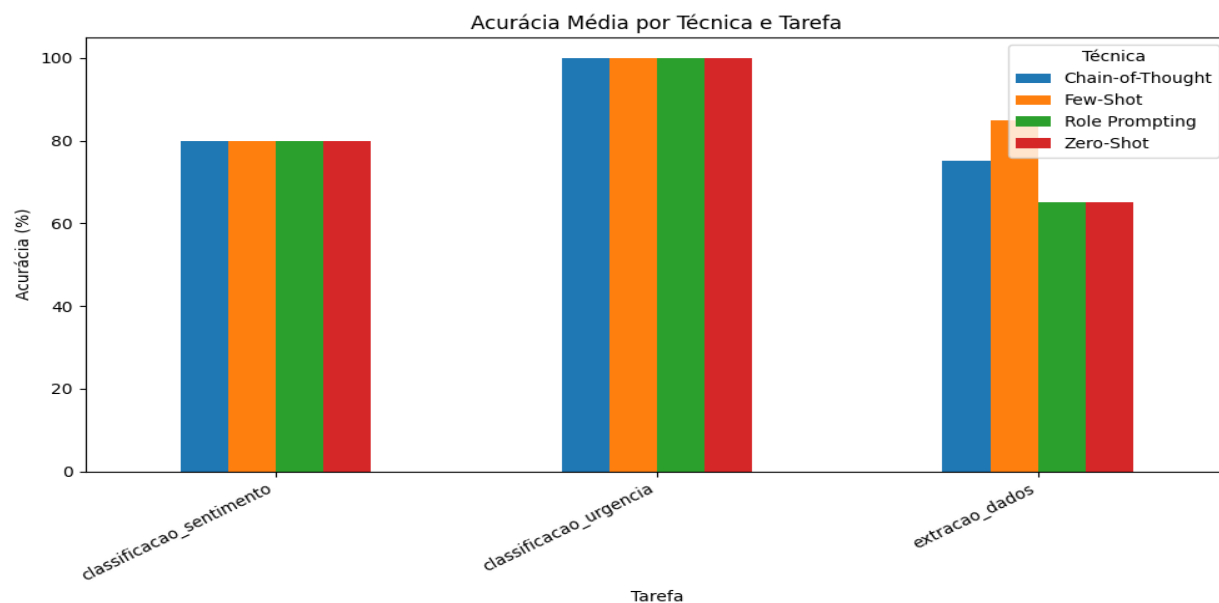


Figura 1 - O gráfico demonstra que Few-Shot obteve melhor desempenho em extração estruturada, enquanto todas as técnicas tiveram desempenho semelhante em classificação de urgência.

Custo médio em tokens: Chain-of-Thought e Role Prompting consumiram mais tokens devido ao contexto adicional e raciocínio explícito.

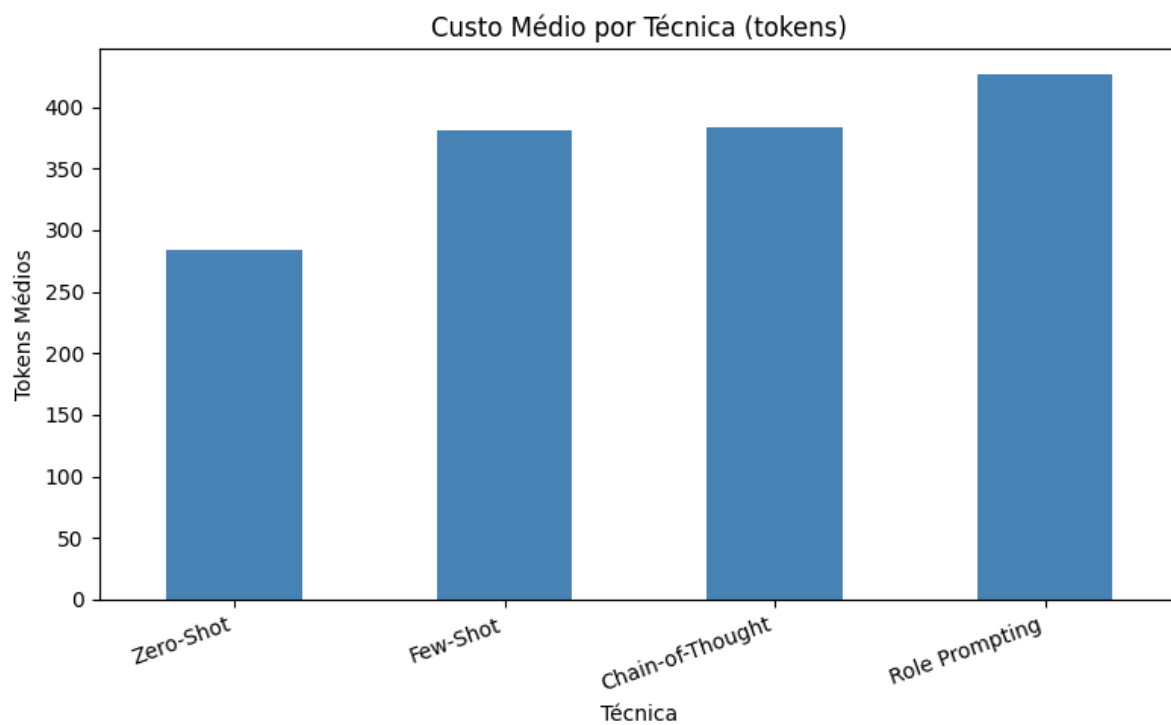


Figura 2 - Custo médio em tokens

Consistência por temperatura: O modelo manteve comportamento estável mesmo em temperaturas diferentes, indicando alta consistência para tarefas classificatórias.

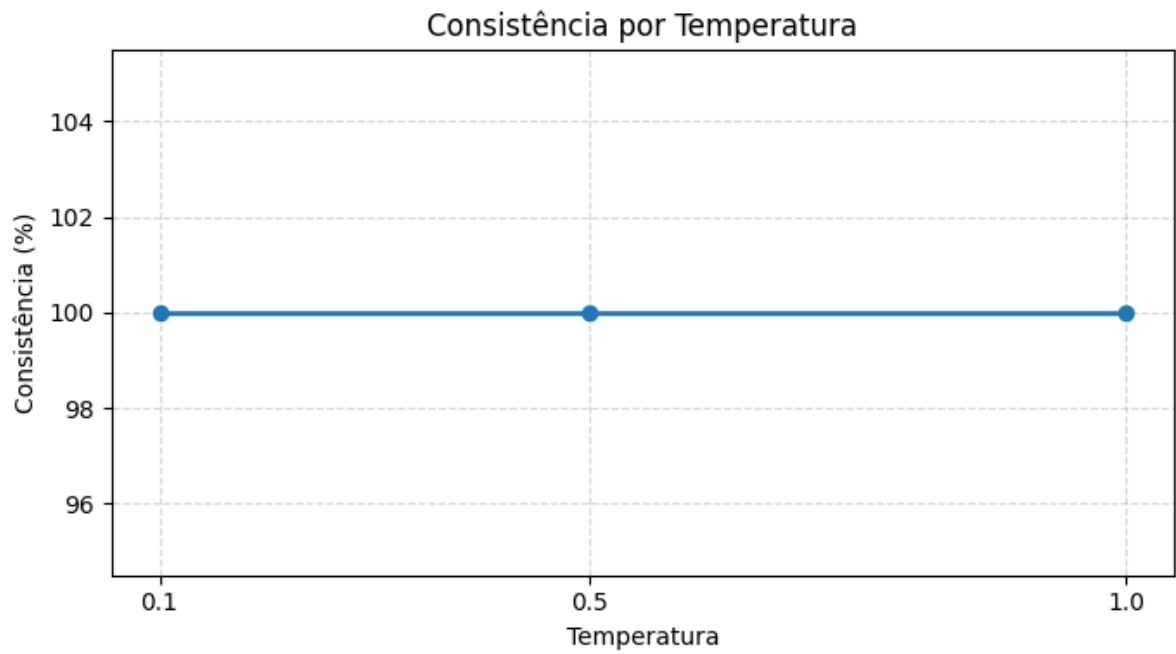


Figura 3 - Consistência por temperatura

6. GUIA DE BOLSO

Técnica	Melhor uso	Custo
Zero-Shot	tarefas simples	baixo
Few-Shot	extração estruturada	médio
Chain-of-Thought	raciocínio complexo	médio-alto
Role Prompting	respostas contextualizadas	alto

7. REFLEXÃO FINAL

O projeto demonstrou como pequenas mudanças na construção dos prompts impactam diretamente a precisão, o custo computacional, a consistência e a qualidade das respostas. A técnica que apresentou os melhores resultados foi o Few-Shot, principalmente em tarefas de extração de dados estruturados, pois exemplos explícitos ajudam o modelo a reproduzir corretamente formatos JSON. O Chain-of-Thought também apresentou resultados fortes em tarefas analíticas, porém com maior

custo em tokens. Entre os principais aprendizados, destacou-se que prompts maiores não garantem maior precisão, exemplos são extremamente úteis em tarefas estruturadas, modelos modernos são altamente consistentes em tarefas simples e a engenharia de prompts influencia diretamente tanto o desempenho quanto o custo. Como melhorias futuras, o projeto pode evoluir com a utilização de datasets maiores, implementação de cache de respostas, criação de um dashboard interativo, integração de múltiplos modelos LLM e aplicação de avaliações semânticas avançadas.

8. CONCLUSÃO

O SentinelPrompt implementou com sucesso um ambiente completo de experimentação em Prompt Engineering utilizando LLMs reais via Ollama. O sistema executou tarefas reais, comparou técnicas automaticamente, gerou métricas quantitativas e produziu gráficos de análise. Os resultados mostraram que cada técnica possui vantagens específicas dependendo do tipo de tarefa executada. O projeto também demonstrou a importância da engenharia de prompts para melhorar precisão, reduzir custo e aumentar consistência em aplicações reais de IA generativa.